

Advanced programming in the Linux environment

Book Proposal

by
Marco Bruno

Proposal contents

Description.....3

About the authors.....4

Table of contents.....5

Annotated table of contents.....18

Sample Chapter.....20

The Competition.....47

Contact Information.....48

Description

Advanced Programming in the Linux Environment offers a comprehensive and rigorously structured exploration of system programming in C on Linux. Rooted in the authoritative Linux manual pages, the book organizes a vast and complex body of knowledge into a coherent progression, guiding readers from fundamental concepts to advanced system-level techniques.

The work is designed to function both as an academic text and as a professional reference. Its primary objective is to develop a deep, operational understanding of system programming, enabling readers to confidently apply concepts in study, teaching, and real-world software development. By combining theoretical clarity with practical implementation, the book addresses the needs of students, educators, and experienced programmers alike.

C programming on Linux remains the foundation for building user applications, system utilities, services, network software, and libraries. This book provides an extensive and detailed treatment of these domains through carefully documented code examples, in-depth technical analysis, and systematic testing. Each concept is illustrated through real, executable programs, ensuring both clarity and practical relevance.

A defining feature of the book is its close integration with the Linux manual pages. For each library function, multiple usage examples are presented, transforming reference material into an accessible and application-oriented resource. This methodology bridges the gap between documentation and practice, enabling readers to move seamlessly from understanding to implementation.

Beyond covering the standard topics expected in works of this kind, the book distinguishes itself by addressing advanced and often underrepresented areas, including recent developments in the Linux ecosystem. It thus serves as a complete and up-to-date reference for modern system programming in C.

Particular emphasis is placed on understanding how software operates at the system level. Programs are not only written and explained, but also analyzed using diagnostic and debugging tools such as *strace* and *gdb*. Readers are guided through the lifecycle of a program, from loading into memory to execution, interaction with system components, and termination, gaining insight into the underlying mechanisms of the operating system.

The book covers all core areas of system programming, including file I/O, file and directory interfaces, environment variables and system databases, memory management, processes and threads, interprocess communication (IPC), and terminal handling. Advanced topics such as asynchronous I/O, the ELF binary format, UNIX domain sockets, multithreaded programming, system integration, and PAM modules are examined in depth.

Each chapter concludes with a dedicated *Theory and Insights* section, offering deeper analysis and conceptual discussion to reinforce learning and encourage critical understanding of the material.

Combining breadth, depth, and practical focus, *Advanced Programming in the Linux Environment* stands as a definitive resource for mastering system programming in C on Linux.

About the authors

Marco Bruno

Marco Bruno is a software developer and educator with over twenty years of experience in information technology and professional training. He began his career as a system administrator, working extensively with UNIX and Linux systems, where he developed strong expertise in system-level operations.

Over time, he specialized in C programming on Linux, focusing on system programming and low-level software development. He has taught these subjects through private courses and technical training programs, combining practical experience with a clear and methodical teaching approach. He has also gained significant international working experience, collaborating with companies and professionals across different countries and technical environments. This exposure has contributed to a broad and adaptable perspective on software development practices and system architectures.

Throughout his career, Mr. Bruno has developed software for a wide range of companies, working across multiple domains, including system software, user applications, network programming, and web development. His experience spans several widely used programming languages, including C, Python, and Java.

He combines extensive industry experience with a strong ability to both develop complex software systems and effectively teach their underlying principles.

Table of contents

Chapter 0 – Tools

- 0.1. Code, functions and commands
- 0.2. Compilation
- 0.3. Build details
- 0.4. The GNU C Compiler
 - 0.4.1 The linker
 - 0.4.2 Executable files and cross-compiling
- 0.5. The standard library and the headers
- 0.6. Make
- 0.7. Autoconf
- 0.8. Gdb
- 0.9. The Linux File System
- 0.10. Documentation
 - 0.10.1 `gman` and `gtkman`
 - 0.10.2 `whatis`
 - 0.10.3 `info`
 - 0.10.4 `apropos`
 - 0.10.5 The *glibc* documentation
- 0.11. The source code
- 0.12. Error handling and exit functions
- 0.13. The Standards
- 0.14. The GNU General Public License
- 0.15. Theory and insights
 - 0.15.1 The GNU C Compiler: `gcc`
 - 0.15.2 The GNU Debugger: `gdb`
 - 0.15.3 Bytes order
 - 0.15.4 Static and dynamic linking
- List of tables
- List of figures

Chapter I – File I/O

- I.1 Reading files
 - I.1.1 Opening files: `open` and `close`
 - I.1.2 Reading from file: `read`
 - I.1.3 Writing to file: `write`
 - I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`
 - I.1.5 Reading one character at a time: `fgetc` and `fputc`
 - I.1.6 Reading one line at a time: `fgets` and `fputs`
- I.2 Opening files
 - I.2.1 The `fdopen` and `fileno` functions
 - I.2.2 The `freopen` function

- I.2.3 The open flags
- I.3 Writing files
- I.4 Setting file position
 - I.4.1 The `lseek` function
 - I.4.2 Getting and setting the position: `fgetpos` and `fsetpos`
 - I.4.3 The `fseeko` and `ftello` functions
- I.5 Appending to files
- I.6 Creating files
 - I.6.1 The open `O_EXCL` flag
- I.7 Truncating files
- I.8 Reading and writing at a given offset
- I.9 Duplicating a file descriptor
 - I.9.1 The `dup2` function
 - I.9.2 The `dup3` function
 - I.9.3 Setting the `O_CLOEXEC` flag
 - I.9.4 `dup` errors
- I.10 Synchronizing the data
 - I.10.1 The `sync` function
 - I.10.2 Synchronization errors
- I.11 File flags
 - I.11.1 Duplicating a file descriptor
 - I.11.2 File descriptor flags
 - I.11.3 File status flags
 - I.11.4 File descriptors and open file description
- I.12 Multiplexed I/O
 - I.12.1 The `poll` function
 - I.12.2 The `ppoll` function
 - I.12.3 The `epoll` interface
- I.13 Binary I/O
- I.14 Error handling
 - I.14.1 The `errno` macro
 - I.14.2 The `strerror*` functions
 - I.14.3 The *end-of-file* and error indicators
 - I.14.4 The `err` and `warn` functions
 - I.14.5 The *glibc* error reporting functions
 - I.14.6 The `program_invocation_name` variable
- I.15 Buffered I/O
 - I.15.1 Buffering types
 - I.15.2 The interface to `stdio` `FILE` structure
 - I.15.3 The `fwide` function
- I.16 Memory streams
- I.17 Formatted I/O
 - I.17.1 `printf` and `scanf`
 - I.17.2 Other `printf*` functions
- I.18 System calls and library functions

- I.19 Scatter-Gather I/O
 - I.19.1 The `readv` and `writew` functions
 - I.19.2 The `preadv` and `pwritev` functions
- I.20 Unreading
- I.21 Formatted I/O with variable number of arguments
 - I.21.1 `vprintf`
 - I.21.2 `vscanf`
 - I.21.3 `sscanf`
- I.22 Temporary files
 - I.22.1 The `mkstemp*` family functions
 - I.22.2 The `tmpfile` function
 - I.22.3 Temporary files with `open`
 - I.22.4 Temporary directories: `mkdtemp`
- I.23 Advices
- I.24 Asynchronous I/O
 - I.24.1 POSIX AIO
 - I.24.2 Multiple I/O requests in one call
 - I.24.3 The `aio_init` function of the `real-time` library
 - I.24.4 Linux AIO – `io_uring`
 - I.24.5 The `liburing` library
 - I.24.6 The `io_uring` system calls
- I.25 File locking
 - I.25.1 Advisory record locking
 - I.25.2 Opening file description locks
 - I.25.3 The locks on the system
 - I.25.4 The `flock` function
 - I.25.5 The `lockf` function
- I.26 Stream locking
 - I.26.1 Locking `FILE` for `stdio`
 - I.26.2 Nonlocking `stdio` functions
 - I.26.3 Nonlocking functions for *wide-characters*
- I.27 Pipes and FIFO
 - I.27.1 The `pipe` function
 - I.27.2 The `popen` function
 - I.27.3 Two-way communication
 - I.27.4 FIFOs
 - I.27.6 Pipes tunable values
 - I.27.7 I/O on pipes and FIFOs
 - I.27.8 Pipes and formatted I/O
 - I.27.9. The `splice` and `vmsplice` functions
 - I.27.10 Duplicating pipe data
- I.28 `fcntl` commands
 - I.28.1 Managing signals
 - I.28.2 Leases
 - I.28.3 File and directory change notification: `dnotify`
 - I.28.4 Changing the capacity of a pipe

- I.28.5 File Sealing
- I.28.6 File read/write hints
- I.29 File allocation
- I.30 Theory and insights
 - I.30.1 I/O
 - I.30.2 File descriptors
 - I.30.3 File sizes data types
 - I.30.4 Positioning data types
 - I.30.5 Multibyte representation conversion
 - I.30.6 Functions for *wide characters*
 - I.30.7 Reading with `sscanf`
 - I.30.8 Writing on memory streams
 - I.30.9 The `getopt` function
 - I.30.10 Synchronous and Asynchronous I/O
 - I.30.11 The `aiomonitor.c` program
 - I.30.12 The `io_uring_sqe` structure
 - I.30.13 The functions of `liburing`
 - I.30.14 The program from the `io_uring` manual page
 - I.30.15 Pipes and `O_ASYNC`
 - I.30.16 Feature Test Macros
 - I.30.17 Copying files with `copy_file_range`
 - I.30.18 Copying files with `sendfile`

List of tables

List of figures

Chapter II – Files and directories

II.1 Files

- II.1.1 Stat of files: `stat`, `fstat` and `lstat`
- II.1.2 File permissions: `umask`
- II.1.3 Checking file accessibility: `access` and `faccessat`
- II.1.4 Changing file permissions: `chmod` and `fchmod`
- II.1.5 Changing file ownership: `chown`, `fchown` and `lchown`
- II.1.6 Creating and removing links: `link`, `linkat` and `unlink`
- II.1.7 Renaming files and directories: `rename` and `renameat`
- II.1.8 Symbolic links: `symlink` and `readlink`
- II.1.9 File times: `utimensat`, `futimens`, `utime`, `utimes`, `futimes`, `lutimes`
- II.1.10 I-node flags: `ioctl`
- II.1.11 Immutable files
- II.1.12 Extended attributes: `setxattr`, `getxattr`, `removexattr`, `listxattr`

II.2 Directories

- II.2.1 Creating and removing directories: `mkdir`, `mkdirat` and `rmdir`
- II.2.2 Open and reading directories: `opendir`, `fdopendir` and `readdir`
- II.2.3 Closing directories: `closedir`

- II.2.4 Scanning a directory: `scandir` and `scandirat`
- II.2.5 Seeking a directory: `telldir`, `seekdir` and `rewinddir`
- II.2.6 Directory file descriptor: `dirfd`
- II.2.7 File tree walk: `nftw`
- II.2.8 Current working directory: `getcwd`, `chdir` and `fchdir`
- II.2.9 Process root directory: `chroot`
- II.3 Pathname and files configuration
 - II.3.1 Resolving paths and links: `realpath` and `canonicalize_file_name`
 - II.3.2 Working with pathnames: `dirname` and `basename`
 - II.3.3 Pathnames and filenames configuration: `fpathconf` and `pathconf`
 - II.3.4 Getting configuration at runtime with `sysconf`
 - II.3.5 Getting configuration dependent string variables: `confstr`
 - II.3.6 Getting configuration from the CLI with `getconf`
 - II.3.7 Summary
- II.4 Device files
 - II.4.1 Major and minor numbers: `makedev`, `major` and `minor`
 - II.4.2 Creating a device file: `mknod` and `mknodat`
 - II.4.3 Creating FIFO: `mkfifo` and `mkfifoat`
 - II.4.4 The random number generator: `getrandom` and `getentropy`
- II.5 Access control list
- II.6 Monitoring file events
 - II.6.1 The *inotify* interface
 - II.6.2 The *fanotify* interface
- II.7 Capabilities
 - II.7.1 File capabilities
 - II.7.2 The *Bounding Set*
 - II.7.3 The *Securebits* flags
 - II.7.4 Listing the contents of the capabilities sets
 - II.7.5 Capabilities data types
 - II.7.6 Child capabilities
- II.8 The Linux file system
 - II.8.1 The *ext4* layout
 - II.8.2 Mounting a filesystem: `mount` and `umount`
 - II.8.3 *ext4* filesystem operations
 - II.8.4 The *swap* area: `swapon` and `swapoff`
 - II.8.5 The `proc` file system
 - II.8.6 The `tmpfs` file system
 - II.8.7 The `sysfs` file system
 - II.8.8 Getting filesystem statistics : `statfs` and `statvfs`
- II.9 Quotas and resource limits
 - II.9.1 Managing quotas
 - II.9.2 Resource limits: `getrlimit`, `setrlimit` and `prlimit`
- II.10 Theory and insights
 - II.10.1 Getting file permissions with `stat`
 - II.10.2 Setting *umask*

- II.10.3 Unlinking a file while a process has an open descriptor
- II.10.4 Scanning directories with filters
- II.10.5 The `openat` and `openat2` functions
- II.10.6 `name_to_handle` and `open_by_handle_at`
- II.10.7 Extended stat of files: `statx`
- II.10.8 Changing root directory
- II.10.9 *Inotify*
- II.10.10 *Inotify* and *Fanotify*
- II.10.11 *ext4* File attributes
- II.10.12 *ext4* Journaling
- II.10.13 Mount namespaces
- II.10.14 The `unshare` function and the *namespaces*
- II.10.15 The `setns` function
- II.10.16 Changing the properties of a mount
- II.10.17 ID-mapped mounts
- II.10.18 Mount information
- II.10.19 New mount API
- II.10.20 The *Virtual File System*

List of tables

List of figures

Chapter III – Environment and system databases

III.1 Time functions

- III.1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`
- III.1.2 Setting time with `clock_gettime` and `clock_settime`
- III.1.3 Clocks on Linux
- III.1.4 Setting a new time: `tm`
- III.1.5 Time conversion functions
- III.1.6 Formatting date and time: `strftime` and `strptime`
- III.1.7 Time conversion: `getdate`

III.2 The environment

- III.2.1 Working with environment variables: `getenv` and `setenv`
- III.2.2 Environment during an `execve`

III.3 Locale

- III.3.1 Setting locale: `setlocale`
- III.3.2 Working with locale: `localeconv`, `newlocale` and `uselocale`
- III.3.3 Duplicating a locale object: `duplocale`

III.4 Users database: `/etc/passwd`

- III.4.1 Users and groups files
- III.4.2 The `/etc/passwd` file
- III.4.3 Working with the users database : `getpwent` and `setpwent`
- III.4.4 Working with records : `fgetpwent` and `putpwent`

III.5 Groups database: `/etc/group`

- III.5.1 Reading the groups database: `getgrnam` and `getgrgid`

- III.5.2 Working with the database: `getgrent` and `setgrent`
- III.5.3 Getting the groups list: `getgrouplist`
- III.5.4 Supplementary groups: `getgroups`, `setgroups` and `initgroups`
- III.5.5 Real and effective group ID: `getgid`, `getegid`, `setgid`, `setegid`
- III.5.6 Saved Set-User-Id and Set-Group-Id
- III.5.7 Checking if a process is in a group: `group_member`
- III.6 Users functions
 - III.6.1 Setting user ID: `getuid`, `geteuid`, `setuid` and `seteuid`
 - III.6.2 `getresgid` and `getresuid`
- III.7 Password database: `/etc/shadow`
 - III.7.1 Creating encrypted passphrase: `crypt`
 - III.7.2 Generating a salt: `crypt_gensalt`
 - III.7.3 Hiding the input with BSD `readpassphrase`
- III.8 Login
 - III.8.1 Working with the `utmp` file: `getutent`, `getutid`, `setutent`
 - III.8.2 The login: `login` and `logout`
- III.9 Hosts, networks, protocols, services databases
 - III.9.1 The hosts database: `/etc/hosts`
 - III.9.2 The networks database: `/etc/networks`
 - III.9.3 The protocols database: `/etc/protocols`
 - III.9.4 The services database: `/etc/services`
- III.10 System identification
 - III.10.1 Getting system information: `uname`
 - III.10.2 Getting and setting hostname: `gethostname` and `sethostent`
- III.11 Pluggable Authentication Modules
 - III.11.1 PAM Authentication: `pam_authenticate`
 - III.11.2 PAM Session: `pam_open_session` and `pam_close_session`
 - III.11.3 Process environment and PAM environment: `pam_getenv`, `pam_putenv`
 - III.11.4 PAM items: `pam_get_item` and `pam_set_item`
 - III.11.5 Changing password: `pam_chauthtok`
- III.12 Theory and insights
 - III.12.1 The Process Execution Environment
 - III.12.2 The `intmax_t` and `uintmax_t` data types
 - III.12.3 Using `initgroups` to set up the supplementary groups
 - III.12.4 Command line tools for the groups
 - III.12.5 The *Name Service Switch*
 - III.12.6 Command line tools for the users
 - III.12.7 The login process
 - III.12.8 PAM functions summary
- List of tables
- List of figures

Chapter IV – Memory

IV.1 Executable files

- IV.1.1 The *Executable and Linkable Format* (ELF)
- IV.1.2 The ELF Header
- IV.1.3 Section and Segments
- IV.1.4 The symbol table
- IV.1.5 The Relocation table
- IV.1.6 The Dynamic section
- IV.1.7 The Note section
- IV.1.8 Reading ELF
- IV.1.9 Loading ELF into memory
- IV.2 Start and end of a program
 - IV.2.1 The `main` function
 - IV.2.2 The `_exit` and `_Exit` functions
 - IV.2.3 Normal process termination
 - IV.2.4 `at_exit` and `on_exit`
 - IV.2.5 Process termination and exit status
 - IV.2.6 Aborting a program: `abort`
- IV.3 Command line arguments
 - IV.3.1 Parsing options with `getopt`
 - IV.3.2 Parsing options with `getopt_long`
 - IV.3.3 The `getsubopt` function
 - IV.3.2 The *argp* interface
- IV.4 Memory allocation
 - IV.4.1 Memory allocation functions
 - IV.4.2 Processes and memory
 - IV.4.3 The GNU Allocator
 - IV.4.4 The `malloc_trim` function
 - IV.4.5 The `alloca` function
 - IV.4.6 Allocating aligned memory
 - IV.4.7 The `getpagesize` function
- IV.5 Memory mapped I/O
 - IV.5.1 The `mmap` function
 - IV.5.2 Private file mapping
 - IV.5.3 Shared file mapping
 - IV.5.4 Anonymous mapping
 - IV.5.5 Extending a mapped file
 - IV.5.6 Remapping memory
- IV.6 Memory functions
 - IV.6.1 The `memcpy` function
 - IV.6.2 The `memmove` function
 - IV.6.3 Memory initialization: `memset`
 - IV.6.4 Memory search: `memchr` and `memmem`
 - IV.6.5 Comparing memory: `memcmp`
 - IV.6.6 Obfuscating memory: `memfrob`
- IV.7 Protect the memory
 - IV.7.1 The `mprotect` function
 - IV.7.2 Memory protection keys

IV.8	Locking memory
IV.8.1	The <code>mlock</code> function
IV.8.2	The <code>mincore</code> function
IV.9	RAM-Backed Files and Secret Memory
IV.9.1	Anonymous RAM-backed file: <code>memfd_create</code>
IV.9.2	Private anonymous RAM-backed memory: <code>memfd_secret</code>
IV.10	Memory advice
IV.10.1	The <code>madvise</code> function
IV.10.2	The <code>posix_madvise</code> function
IV.10.3	Remote memory advice: <code>process_madvise</code>
IV.11	Nonlocal gotos
IV.11.1	The <code>setjmp</code> and <code>longjmp</code> functions
IV.11.2	The <code>sigsetjmp</code> and <code>siglongjmp</code> functions
IV.12	Core dump file
IV.13	Dynamically loaded objects
IV.13.1	Opening a shared object: <code>dlopen</code>
IV.13.2	Obtaining the address of a symbol: <code>dlsym</code>
IV.13.4	Obtaining information about a loaded object: <code>dlinfo</code>
IV.13.5	Translating address to symbolic information: <code>dladdr</code>
IV.13.6	Walking through a list of shared objects: <code>dl_iterate_phdr</code>
IV.14	Theory and insights
IV.14.1	The dynamic linker
IV.14.2	The C Runtime
IV.14.3	Memory allocation parameters
IV.14.4	<code>malloc</code> related functions
IV.14.5	Heap consistency checking
IV.14.6	Mapping to a specific address
IV.14.7	Memory-management terms glossary
IV.14.8	Getting information about physical pages: <code>get_phys_pages</code>
IV.14.9	Getting the number of processors: <code>get_nprocs</code>
IV.14.10	Querying the cache: <code>cachestat</code>
IV.14.11	Non-Uniform Memory Architecture: <code>numa</code>
	List of tables
	List of figures

Chapter V – Processes

V.1	Processes.....
V.1.1	Get the process identifier: <code>getpid</code> and <code>getppid</code>
V.1.2	Create a process: <code>fork</code>
V.1.3	Create a process blocking the parent: <code>vfork</code>
V.1.4	Block the execution: <code>wait</code> , <code>waitpid</code> and <code>waitid</code>
V.1.5	The <code>wait3</code> and <code>wait4</code> functions
V.1.6	Process file descriptors
V.1.7	Create child processes: <code>clone</code> and <code>clone3</code>

- V.1.8 Compare processes: `kcmp`
- V.1.9 Execute a shell command: `system`
- V.1.10 Execute a program: `execve`
- V.1.11 Execute a program via file descriptor: `fexecve`
- V.1.12 The `exec` family functions
- V.1.13 Control a process: `prctl`
- V.1.14 Process data transfer: `process_vm_read`
- V.1.15 Secure Computing state of the process: `seccomp`
- V.1.16 Secure Computing state of the process: `libseccomp`
- V.1.17 Seccomp user-space notification mechanism: `seccomp_unotify`
- V.1.18 Seccomp user-space notification mechanism with `libseccomp`
- V.1.19 Process accounting: `acct`
- V.1.20 Process scheduling: `nice` and `setpriority`
- V.2 Process groups, sessions and terminal
 - V.2.1 Process groups: `getpgid` and `setpgid`
 - V.2.2 Sessions: `setsid` and `getsid`
 - V.2.3 Controlling terminal: `tcsetpgrp`
 - V.2.4 Job control
 - V.2.5 Identifying the controlling terminal: `ctermid` and `ttyname`
 - V.2.6 Orphaned process groups
- V.3 Signals
 - V.3.1 Signal overview
 - V.3.2 Signal details
 - V.3.3 Describe signals: `strsignal` and `psignal`
 - V.3.4 Signal handling: `sigaction`
 - V.3.5 Signal set operations: `sigemptyset` and `sigfillset`
 - V.3.6 Signal handlers
 - V.3.7 Send signals to the caller: `raise`
 - V.3.8 Send signals to another process: `kill` and `killpg`
 - V.3.9 Send signals to a specific thread: `tgkill` and `pthread_kill`
 - V.3.10 Send real-time signals: `sigqueue` and `pthread_sigqueue`
 - V.3.11 Set alarm clock: `alarm`
 - V.3.12 Block signals: `sigprocmask`
 - V.3.13 Check for pending signals: `sigpending`
 - V.3.14 Handling signals with nonlocal jumps
 - V.3.15 Wait for signals: `pause` and `sigsuspend`
 - V.3.16 Wait for signals: `sigwait` and `sigwaitinfo`
 - V.3.17 Signal file descriptor: `signalfd`
 - V.3.18 Alternate signal stack: `sigaltstack`
 - V.3.19 Sleeping: `sleep`, `nanosleep`, `usleep`, `clock_nanosleep`
- V.4 Timers
 - V.4.1 Interval timers: `getitimer` and `setitimer`
 - V.4.2 POSIX timers: `timer_create`
 - V.4.3 Arm and disarm timers: `timer_gettime` and `timer_settime`
 - V.4.4 Overruns and deletion: `timer_getoverrun` and `timer_delete`

V.4.5 CPU-time clocks: `clock_gettime`, `pthread_getcpuclockid`

V.5 Threads

V.5.1 POSIX threads: `pthread`

V.5.2 Thread ID: `gettid`, `pthread_self` and `pthread_equal`

V.5.3 Create threads: `pthread_create` and `pthread_join`

V.5.4 Thread attributes: `pthread_attr_t` and `pthread_attr_destroy`

V.5.5 Terminate threads: `pthread_exit` and `pthread_cancel`

V.5.6 Clean-up handlers: `pthread_cleanup_push`, `pthread_cleanup_pop`

V.5.7 Send signal to threads: `pthread_kill` and `pthread_sigmask`

V.5.8 Thread-specific data: `pthread_key_create`

V.5.9 Mutexes

V.5.10 Mutex attributes

V.5.11 Timed mutex locking: `pthread_mutex_timedlock`

V.5.12 Read-Write Locks

V.5.13 Condition variables

V.5.14 Spin locks

V.5.15 Barriers

V.5.16 Fork handlers: `pthread_atfork`

V.5.17 Semaphores

V.5.18 Futexes

V.5.19 Thread name: `pthread_setname_np`

V.6 Daemon processes and logging

V.6.1 Daemon overview

V.6.2 The `daemon` function

V.6.3 The `systemd` service API: `sd-daemon.h`

V.6.4 Logging: `syslog`

V.6.5 Logging: `sd-journal`

V.7 Theory and insights

V.7.1 Values for `flags` and `clone_args.flags`

V.7.2 `prctl` operations

V.7.3 The `seccomp_unotify(2)` manual page program

V.7.4 CPU sets macros

V.7.5 Linux control groups: `cgroup`

V.7.6 Trace a process: `ptrace`

V.7.7 Process self-debugging: `backtrace`

V.7.8 Thread attributes functions

V.7.9 Patterns of synchronization mechanisms

List of tables

List of figures

Chapter VI – Interprocess Communication and Terminals

VI.1 Introduction

VI.2 POSIX Message Queues

VI.2.1 Create a message queue: `mq_open`, `mq_close`, `mq_unlink`

- VI.2.2 Send and receive messages: `mq_send` and `mq_receive`
- VI.2.3 Message queue attributes: `mq_getattr` and `mq_setattr`
- VI.2.4 Message queue notification: `mq_notify`
- VI.2.5 Notification via thread
- VI.2.6 Notification via signal
- VI.3 POSIX Named Semaphores
 - VI.3.1 Create and delete semaphores: `sem_open`, `sem_close`, `sem_unlink`
 - VI.3.2 Process-local mutual exclusion
 - VI.3.3 Interprocess mutual exclusion
 - VI.3.4 Two-way synchronization pattern
- VI.4. POSIX Shared Memory
 - VI.4.1 Overview
 - VI.4.2 Producer-consumer pattern
 - VI.4.3 Client-server pattern
- VI.5 Sockets
 - VI.5.1 Overview
 - VI.5.2 Creating sockets: `socket` and `shutdown`
 - VI.5.3 Byte order conversion functions
 - VI.5.4 Addressing: `getaddrinfo` and `getnameinfo`
 - VI.5.5 Address formats: `inet_ntop` and `inet_pton`
 - VI.5.6 Setting the socket address: `bind`
 - VI.5.7 Receiving connections: `listen` and `accept`
 - VI.5.8 Connecting to sockets: `connect`
 - VI.5.9 Socket information: `getsockname`, `getpeername`, `getsockopt`
 - VI.5.10 Data transmission: `send` and `recv`
 - VI.5.11 Connection-oriented client-server uptime service
 - VI.5.12 Non-blocking sockets and asynchronous I/O
 - VI.5.13 Out-of-band data: `socketatmark`
 - VI.5.14 Sending and receiving multiple messages: `sendmmsg` and `recvmmsg`
- VI.6 UNIX Domain Sockets
 - VI.6.1 Unnamed UNIX domain sockets: `socketpair`
 - VI.6.2 Named UNIX domain sockets: `sockaddr_un`
 - VI.6.3 Datagram sockets: `SOCK_DGRAM`
 - VI.6.4 Stream-oriented sockets and message-oriented sockets
 - VI.6.5 Abstract sockets
 - VI.6.6 Socket options and ancillary messages: `cmsg`
 - VI.6.7 Passing file descriptors within a single process
 - VI.6.8 Passing file descriptors between parent and child processes
 - VI.6.9 Passing file descriptors and credentials between unrelated processes
 - VI.6.10 A secure file access service
 - VI.6.11 Sequenced-packet sockets: `SOCK_SEQPACKET`
- VI.7 Terminal I/O.....
 - VI.7.1 Identifying terminal file descriptors: `isatty`
 - VI.7.2 Terminal properties: `termios`
 - VI.7.3 Retrieving and changing terminal settings: `tcgetattr` and `tcsetattr`

- VI.7.4 Canonical, noncanonical and raw mode
- VI.7.5 Line control
- VI.7.6 Line speed
- VI.7.7 Special characters
- VI.7.8 Window terminal size
- VI.7.9 Command-line utilities
- VI.7.10 `ncurses`
- VI.8 Pseudo terminals
 - VI.8.1 Overview
 - VI.8.2 UNIX 98 pseudoterminal
 - VI.8.3 `script` and `expect`
 - VI.8.3 Opening pseudoterminal: `getpt` and `posix_openpt`
 - VI.8.4 Unlocking pseudoterminal: `grantpt` and `unlockpt`
 - VI.8.5 Getting pseudoterminal name: `ptsname`
 - VI.8.6 Pseudoterminal pair: `openpty`, `forkpty` and `loginpty`
- VI.9 Theory and insights
 - VI.9.1 POSIX Message queues limits files
 - VI.9.2 System V interprocess communication
 - VI.9.3 Network interfaces functions
 - VI.9.4 Virtual console: `vcs`
 - VI.9.5 Serial terminal lines: `ttyS`
- List of tables
- List of figures

Annotated table of contents

Chapter 0 — Tools

This introductory chapter presents the essential tools commonly used for software development in a Linux environment, along with an overview of the standards that define software behavior and portability. It is intended to provide readers with the necessary background for the topics covered in the rest of the book; those already familiar with these concepts may choose to skip it.

The chapter covers the compilation process and the use of key development tools such as *gcc*, *make*, and *autoconf*, as well as debugging with *gdb*. It also introduces the source code and how it can be obtained and organized in a Linux system. In addition, it outlines the most widely used software standards at a high level and includes a simple introduction to the GNU General Public License (GPL) and its role in software development and distribution.

Chapter 1 — File I/O

This chapter introduces file input/output operations, starting from fundamental system calls such as *open*, *read*, *write*, and *close*, and progressing to more advanced mechanisms. Topics include file truncation and appending, file descriptor duplication, I/O multiplexing, buffering strategies, asynchronous I/O (both POSIX and Linux-specific), file locking, and interprocess communication through pipes and FIFOs.

Chapter 2 — Files and Directories

This chapter examines the APIs used to interact with the filesystem. It covers file metadata (*stat*), links, permissions, and inode structures, as well as operations on directories such as creation, deletion, and renaming. Additional topics include device files, Linux capabilities, filesystem organization, quotas, and resource limits.

Chapter 3 — Environment and system Databases

This chapter explores the process environment and its interaction with the operating system. It includes functions for handling time and date, environment variables, and localization (locale). It also provides a detailed overview of system databases, such as user and group information, along with login records and system identification (*uname*). The chapter concludes with a dedicated section on Pluggable Authentication Modules (PAM).

Chapter 4 — Memory

This chapter provides an in-depth analysis of memory management in Linux. It covers dynamic memory allocation, memory mapping, memory protection, and locking mechanisms. It also examines dynamically loaded objects and presents a detailed study of the ELF (Executable and Linkable Format), including how programs are loaded into memory and how processes start and terminate.

Chapter 5 — Processes

This chapter focuses on process management and multithreading. It covers process creation and control through *fork*, *clone*, *execve*, and *wait*, as well as security features such as *seccomp*. Additional topics include process groups, sessions, terminal control, signal handling, timers, thread programming, and daemon processes, all supported by practical examples.

Chapter 6 — Interprocess Communication and Terminals

This chapter covers the main IPC mechanisms available in Linux, including POSIX message queues, semaphores, shared memory, and sockets. It also explores UNIX domain sockets in detail,

along with terminal and pseudoterminal interfaces, providing numerous examples to illustrate their use in real-world scenarios.

Sample Chapter

Chapter VI Interprocess Communication and Terminals

VI.1 Introduction

Inter-Process Communication (IPC) refers to mechanisms that allow processes to communicate and coordinate with each other. Communication includes sending and receiving data, sharing resources, and synchronizing execution between processes.

IPC is essential because processes in modern operating systems are typically isolated for safety and stability, and therefore need controlled methods to exchange information.

In Linux, IPC can be implemented using several mechanisms:

- *Pipes*
Used for unidirectional (half-duplex) communication between related processes (e.g., parent and child).
 - *Unnamed pipes*: typically created with `pipe`
 - *Named pipes (FIFOs)*: allow communication between unrelated processes
- *Message Queues*
Allow processes to exchange messages in a structured and asynchronous way. Messages are stored in a queue and can be retrieved in order or based on priority
- *Semaphores*
Used for process synchronization rather than data exchange. They help control access to shared resources and prevent race conditions
- *Shared Memory*
The fastest IPC mechanism, where multiple processes share a common *memory segment*. Processes can read and write directly to this memory with synchronization (e.g., using semaphores), that is required to avoid conflicts
- *Sockets*
Provide communication between processes, either on the same machine or over a network. They are commonly used for client-server communication

We have already examined some IPC mechanisms. In this section, we will focus on named semaphores, message queues, and shared memory, followed by an introduction to sockets.

Message queues, semaphores, and shared memory are part of XSI IPC, a standardized set of IPC mechanisms originally introduced in System V UNIX¹.

1 See the `sysvipc(7)` manual page

VI.2 POSIX Message Queues

Message queues are an IPC mechanism that allow processes or threads to communicate by sending and receiving messages through a queue managed by the operating system. The first implementation of message queues came from System V, but it had several limitations. POSIX message queues were later introduced to provide a simpler API, support message priorities, and offer better integration with modern system features like asynchronous notification.

POSIX message queues are widely used in modern applications. They provide a file-like interface with operations such as `open`, `close`, and `unlink`. Furthermore, they support message priorities directly and integrate seamlessly with I/O multiplexing mechanisms like `select` and `poll` via file descriptors.

Message queues are created and opened using `mq_open`. This function returns a *message queue descriptor* (`mqd_t`), which is used in subsequent operations to refer to the opened queue.

Each message queue is identified by a name of the form `/somename`, which is a null-terminated string of up to `NAME_MAX` (typically 255) characters. The name must begin with a forward slash (`/`) and must not contain any additional slashes. Two processes can communicate using the same queue by passing the same name to `mq_open`.

Messages are sent and received using `mq_send` and `mq_receive`. When a process no longer needs a queue, it closes it using `mq_close`. The queue itself can be removed from the system using `mq_unlink`; when it is no longer required. Queue attributes can be retrieved and, in some cases, modified using `mq_getattr` and `mq_setattr`. Additionally, a process can request asynchronous notification of message arrival on an empty queue using `mq_notify`.

A *message queue descriptor* refers to an *open message queue description*. After a `fork`, the child process inherits copies of its parent's message queue descriptors, and both processes refer to the same underlying open queue. The associated flags (`mq_flags`) are shared between corresponding descriptors.

Each message has an associated *priority*, and messages are delivered to the receiving process in order of highest priority first. Message priorities range from 0 (lowest) to `sysconf(_SC_MQ_PRIO_MAX) - 1` (highest). This value is typically 32768^2 .

Message queues are created in a virtual filesystem that can be mounted using the commands:

```
[linux@ipc] sudo mkdir /dev/mqueue  
[linux@ipc] sudo mount -t mqueue none /dev/mqueue
```

On the mount directory the *sticky bit* is automatically enabled. The message queues on the system can be viewed and manipulated using the commands usually used for files such as `ls` and `rm`. Each file consists in a single line containing information about the queue

```
[linux@ipc] cat /dev/mqueue/mymq  
QSIZE:129 NOTIFY:2 SIGNO:0 NOTIFY_PID:8260
```

2 POSIX only requires support for at least priorities in the range 0 to 31

The fields are:

QSIZE

Number of bytes of data in all messages in the queue

NOTIFY_PID

If this is nonzero, then the process with this PID has used mq_notify to register for asynchronous message notification, the remaining fields describe how notification occurs

NOTIFY

Notification method:

0 → SIGEV_SIGNAL

1 → SIGEV_NONE

2 → SIGEV_THREAD

SIGNO

Signal number to be used for SIGEV_SIGNAL

VI.2.1 Create a message queue: `mq_open`, `mq_close`, `mq_unlink`

The library functions are implemented on top of underlying system calls as listed in the following table.³

Library interface	System call
<code>mq_close</code> (3)	<code>close</code> (2)
<code>mq_getattr</code> (3)	<code>mq_getsetattr</code> (2)
<code>mq_notify</code> (3)	<code>mq_notify</code> (2)
<code>mq_open</code> (3)	<code>mq_open</code> (2)
<code>mq_receive</code> (3)	<code>mq_timedreceive</code> (2)
<code>mq_send</code> (3)	<code>mq_timedsend</code> (2)
<code>mq_setattr</code> (3)	<code>mq_getsetattr</code> (2)
<code>mq_timedreceive</code> (3)	<code>mq_timedreceive</code> (2)
<code>mq_timedsend</code> (3)	<code>mq_timedsend</code> (2)
<code>mq_unlink</code> (3)	<code>mq_unlink</code> (2)

Tab 3. POSIX Message queues library functions

To create/open a message queue we can use the `mq_open`⁴ function.

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqqueue.h>

mqd_t mq_open(const char *name, int oflag);

mqd_t mq_open(const char *name, int oflag, mode_t mode, struct mq_attr
              *attr);

Return: a message queue descriptor, on error (mqd_t) -1 and errno is set
```

The `mq_open` function creates a new message queue or opens an existing queue. The queue is identified by `name`.

The `oflag` argument specifies the flags that control the operation as in `open`, and exactly only one flag must be specified.

³ See the `mq_overview`(7) and `mqqueue.h`(7POSIX) manual pages

⁴ See the `mq_open`(3) manual page

The `mode` argument, in the case of `O_CREAT`, specifies the permissions for the new queue.

The `attr` argument is a structure defined as follows

```
struct mq_attr {
    long mq_flags;      /* Flags (ignored for mq_open()) */
    long mq_maxmsg;     /* Max. # of messages on queue */
    long mq_msgsize;    /* Max. message size (bytes) */
    long mq_curmsgs;    /* # of messages currently in queue ignored for
                        mq_open() */
};
```

When calling `mq_open`, only `mq_maxmsg` and `mq_msgsize` are employed, the other fields are ignored. If `attr` is `NULL`, then the queue is created with implementation-defined default attributes.

A descriptor returned by `mq_open` has the *close-on-exec* flag automatically set.

To close a message queue we can use `mq_close`⁵.

```
#include <mqqueue.h>

int mq_close(mqd_t mqdes);

Returns: 0 on success, -1 on error and errno is set
```

The `mq_close` function closes the message queue descriptor `mqdes`. If the calling process has attached a notification request (`mq_notify`) to this message queue via `mqdes`, then this request is removed.

To remove a message queue we can use `mq_unlink`⁶.

```
#include <mqqueue.h>

int mq_unlink(const char *name);

Returns: 0 on success, -1 on error and errno is set
```

The `mq_unlink` function removes the specified message queue name. The message queue name is removed immediately. The queue itself is destroyed once any other processes that have the queue open close their descriptors referring to the queue.

⁵ See the `mq_close(3)` manual page

⁶ See the `mq_unlink(3)` manual page

VI.2.2 Send and receive messages: `mq_send` and `mq_receive`

To send a message to a message queue we can use the `mq_send`⁷ function, defined as follows.

```
#include <mqueue.h>
#include <time.h>

int mq_send(mqd_t mqdes, const char msg_ptr[msg_len], size_t msg_len,
            unsigned int msg_prio);

int mq_timedsend(mqd_t mqdes, const char msg_ptr[msg_len], size_t
                msg_len, unsigned int msg_prio, const struct timespec
                *abs_timeout);

Return: on success 0, on error -1 and errno is set
```

The `mq_send` function inserts the message pointed to by `msg_ptr` into the message queue identified by the descriptor `mqdes`. The `msg_len` argument specifies the size of the message and must be less than or equal to the queue's `mq_msgsize` attribute. Messages of zero length are permitted.

The `msg_prio` argument specifies the message priority as a nonnegative integer. Messages are ordered in the queue by decreasing priority; messages with the same priority are delivered in FIFO order, with newer messages placed after older ones.

If the queue is full (i.e., the number of messages equals the `mq_maxmsg` limit), the default behavior of `mq_send` is to block until space becomes available or the call is interrupted by a signal. If the `O_NONBLOCK` flag is set for the queue, the call does not block and instead fails immediately with the error `EAGAIN`.

The `mq_timedsend` function behaves similarly to `mq_send`, but allows a timeout to be specified. If the queue is full and `O_NONBLOCK` is not set, the call blocks only until the absolute timeout specified by `abs_timeout` is reached.

If the timeout has already expired at the time of the call, `mq_timedsend` returns immediately without sending the message.

To receive a message from the message queue we can use the `mq_receive`⁸ function, defined as follows

```
#include <mqueue.h>
#include <time.h>
```

⁷ See the `mq_send(3)` manual page

⁸ See the `mq_receive(3)` manual page

```

ssize_t mq_receive(mqd_t mqdes, char msg_ptr[msg_len], size_t msg_len,
                  unsigned int *msg_prio);

ssize_t mq_timedreceive(mqd_t mqdes, char *restrict msg_ptr[msg_len],
                      size_t msg_len, unsigned int *restrict msg_prio,
                      const struct timespec *restrict abs_timeout);

    Return: the number of bytes in the received message, on error -1 and errno

```

The `mq_receive` function retrieves and removes the oldest message with the highest priority from the message queue identified by the descriptor `mqdes`, and stores it in the buffer pointed to by `msg_ptr`. The `msg_len` argument specifies the size of this buffer and must be greater than or equal to the queue's `mq_msgsize` attribute.

If `msg_prio` is not NULL, it is used to return the priority of the received message.

If the queue is empty, the default behavior of `mq_receive` is to block until a message becomes available or the call is interrupted by a signal. If the `O_NONBLOCK` flag is set, the function does not block and instead fails immediately with the error `EAGAIN`.

The `mq_timedreceive` function behaves similarly to `mq_receive`, but allows a timeout to be specified. If the queue is empty and `O_NONBLOCK` is not set, the call blocks only until the absolute timeout specified by `abs_timeout`.

If no message is available and the timeout has already expired at the time of the call, `mq_timedreceive` returns immediately.

The `mq_send.c` and `mq_recv.c` programs use a POSIX message queue. `mq_recv.c` waits for a message to be received, `mq_send.c` sends the message inserted by the user.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mqqueue.h>
4 #include <fcntl.h>
5 #include <sys/stat.h>

6 #define QUEUE_NAME "/myqueue"
7 #define MAX_SIZE 128

8 int main() {
9     mqd_t mq;
10    char buffer[MAX_SIZE];
11    unsigned int priority;
12    struct mq_attr attr;

13    attr.mq_flags = 0;
14    attr.mq_maxmsg = 10;
15    attr.mq_msgsize = MAX_SIZE;
16    attr.mq_curmsgs = 0;

```

```

17  mq = mq_open(Queue_NAME, O_CREAT | O_RDONLY, 0644, &attr);
18  if (mq == (mqd_t) -1) {
19      perror("mq_open");
20      exit(1);
21  }
22  if (mq_receive(mq, buffer, MAX_SIZE, &priority) == -1) {
23      perror("mq_receive");
24      exit(1);
25  }

26  printf("Received message: %s", buffer);
27  printf("Priority: %u\n", priority);
28  mq_close(mq);
29  mq_unlink(Queue_NAME);
30  return 0;
31 }

```

mq_rcv.c Retrieves a message from the queue

At line 6, the name of the message queue is defined. On Linux systems, this queue will appear as an entry under `/dev/mqueue`.

At line 12, a `struct mq_attr` variable is declared to specify the attributes of the message queue.

In lines 13–16, the queue attributes are initialized:

- `mq_maxmsg` is set to 10, defining the maximum number of messages the queue can hold
- `mq_msgsize` is set to 128 bytes, defining the maximum size of each message
- `mq_flags` is set to 0, meaning blocking mode is used by default
- `mq_curmsgs` is initialized to 0 (this field is ignored when creating the queue)

The queue is then created (or opened) with the call

```

17  mq = mq_open(Queue_NAME, O_CREAT | O_RDONLY, 0644, &attr);

```

This call creates the queue `/myqueue` (visible as `/dev/mqueue/myqueue`) if it does not already exist, using the specified permissions and attributes. If the queue already exists, it is simply opened, and the `attr` structure is ignored.

Next, the program attempts to receive a message using

```

22  if (mq_receive(mq, buffer, MAX_SIZE, &priority) == -1) {

```

The arguments are

<code>mq</code>	the message queue descriptor
<code>buffer</code>	the buffer where the received message will be stored
<code>MAX_SIZE</code>	the size of the buffer (which must be at least the queue's <code>mq_msgsize</code>)

`priority` a variable used to retrieve the message priority

Since the queue is opened in blocking mode and no `O_NONBLOCK` flag is specified, this call blocks until a message becomes available. After successfully receiving the message, the program prints its contents and priority.

At line 28, the queue descriptor is closed using `mq_close`

At line 29, the queue is removed from the system using `mq_unlink`. This deletes the queue name, and the queue itself is destroyed once no processes are using it.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <mqqueue.h>
5 #include <fcntl.h>

6 #define QUEUE_NAME "/myqueue"
7 #define MAX_SIZE 128

8 int main() {
9     mqd_t mq;
10    struct mq_attr attr;

11    mq = mq_open(QUEUE_NAME, O_WRONLY, 0644, &attr);
12    if (mq == (mqd_t) -1) {
13        perror("mq_open");
14        exit(1);
15    }

16    char message[MAX_SIZE];
17    printf("Enter message: ");
18    fgets(message, MAX_SIZE, stdin);
19    if (mq_send(mq, message, strlen(message) + 1, 1) == -1) {
20        perror("mq_send");
21        exit(1);
22    }
23    printf("Message sent: %s\n", message);
24    mq_close(mq);
25    return 0;
26 }
```

mq_send.c Sends a message to the queue

At line 6, the name of the message queue is defined. This must match the name used by the receiver so that both processes operate on the same queue.

At line 10, a `struct mq_attr` variable is declared. However, in this program it is not actually used, since the queue is opened without the `O_CREAT` flag. Therefore, the `attr` argument passed to `mq_open` is ignored and could be replaced with `NULL`.

The queue is then opened using:

```
11    mq = mq_open(QUEUE_NAME, O_WRONLY, 0644, &attr);
```

This call opens an existing message queue in write-only mode. It assumes that the queue has already been created (for example, by the receiver). If the queue does not exist, the call fails.

In lines 16–18, the program reads a message from standard input using `fgets` and stores it in the message buffer.

The message is then sent to the queue using:

```
19    if (mq_send(mq, message, strlen(message) + 1, 1) == -1) {
```

The arguments are

- `mq` the message queue descriptor
- `message`: the buffer containing the message
- `strlen(message) + 1` the size of the message (including the null terminator)
- `1` the priority assigned to the message

The message is inserted into the queue and will be delivered according to its priority.

Finally, at line 24, the queue descriptor is closed using `mq_close`.

Let's execute the programs

Shell 1

```
[linux@ipc] ./mq_rcv
```

Shell 2

```
[linux@ipc] ./mq_send
Enter message: hello receiver!
Message sent: hello receiver!
```

Shell 1

```
Received message: hello receiver!
Priority: 1
```

After the sender sends the message, the receiver reads it from the queue and prints its contents.

If we comment out the `mq_unlink` call, we can see the `myqueue` file under `/dev/mqueue`.

```
[linux@ipc] cat /dev/mqueue/myqueue
QSIZE:0    NOTIFY:0    SIGNO:0    NOTIFY_PID:0
```

In this example, the message queue is created by the receiver, while the sender simply uses the existing queue. This is a design choice made for illustrative purposes. In practice, the specific process responsible for creating the queue is less important than ensuring that all participating components can access and use it correctly.

For instance, the queue could instead be created by the sender, which might even send messages before the receiver starts executing. Alternatively, both the sender and the receiver could attempt to create the queue using the `O_CREAT` flag; since the queue is only created if it does not already exist, this approach improves robustness by removing dependencies on process startup order.

In this example, however, creating the queue in the receiver allows us to demonstrate that `mq_receive` blocks when no messages are available, waiting until a message is delivered.

The process that creates the queue defines its attributes, while other processes can retrieve these attributes as needed.

VI.2.3 Message queue attributes: `mq_getattr` and `mq_setattr`

The `mq_getattr` and `mq_setattr` functions⁹ respectively retrieve and modify attributes of the message queue referred to by the message queue descriptor `mqdes`.

```
#include <mqueue.h>

int mq_getattr(mqd_t mqdes, struct mq_attr *attr);

int mq_setattr(mqd_t mqdes, const struct mq_attr *restrict newattr,
               struct mq_attr *restrict oldattr);

Return: on success 0, on error -1 and errno is set
```

The `mq_getattr` and `mq_setattr` functions, respectively retrieve and modify attributes of the message queue referred to by the message queue descriptor `mqdes`.

`mq_getattr` returns an `mq_attr` structure in the buffer pointed by `attr`. In the `mq_attr` structure, the `mq_flags` field contains flags associated with the open message queue description. This field is initialized when the queue is created by `mq_open`. The only flag this field can have is `O_NONBLOCK`.

Also `mq_maxmsg` and `mq_msgsize` are set when the message queue is created. `mq_maxmsg` is the upper limit on the number of messages that may be placed on the queue using `mq_send`. `mq_msgsize` is the upper limit on the size of messages that may be placed on the queue. Both of these fields must have a value greater than zero.

The `mq_curmsgs` field returns the number of messages currently held in the queue.

The `mq_setattr` function sets message queue attributes using information supplied in the `mq_attr` structure pointed to by `newattr`. The only attribute that can be modified is the setting of the `O_NONBLOCK` flag in `mq_flags`, the other fields in `newattr` are ignored. If the `oldattr` field is not `NULL`, then the buffer that it points to is used to return an `mq_attr` structure that contains the same information that is returned by `mq_getattr`.

The `mqgetattr.c` program is taken from the `mq_getattr(2)` manual page. It shows how to retrieve the attributes associated to a message queue created by passing the `attr` argument as `NULL`.

```
1 #include <err.h>
2 #include <fcntl.h>
3 #include <mqueue.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <unistd.h>

7 int main(int argc, char *argv[])
```

⁹ See the `mq_getattr(3)` manual page

```

8 {
9     mqd_t mqd;
10    struct mq_attr attr;

11    if (argc != 2) {
12        fprintf(stderr, "Usage: %s mq-name\n", argv[0]);
13        exit(EXIT_FAILURE);
14    }

15    mqd = mq_open(argv[1], O_CREAT | O_EXCL, 0600, NULL);
16    if (mqd == (mqd_t) -1)
17        err(EXIT_FAILURE, "mq_open");

18    if (mq_getattr(mqd, &attr) == -1)
19        err(EXIT_FAILURE, "mq_getattr");

20    printf("Maximum # of messages on queue: %ld\n", attr.mq_maxmsg);
21    printf("Maximum message size: %ld\n", attr.mq_msgsize);
22    if (mq_unlink(argv[1]) == -1)
23        err(EXIT_FAILURE, "mq_unlink");
24    exit(EXIT_SUCCESS);
25 }

```

mqgetattr.c

The program creates a message queue with the name specified on the command line. When `NULL` is passed as the `attr` argument, the queue is initialized with default attributes. These attributes are subsequently retrieved and printed by the program.

The call

```
18    if (mq_getattr(mqd, &attr) == -1)
```

retrieves the attributes associated with the queue `mqd`.

Before removing the queue, the program prints the values of the `mq_maxmsg` and `mq_msgsize` attributes.

Let's execute

```

[linux@ipc] ./mqgetattr /mynewq
Maximum # of messages on queue: 10
Maximum message size:          8192

```

We can check these two values in the `/proc/sys/fs/mqueue/msg_default` and `/proc/sys/fs/mqueue/msgsize_max` files.

Both `mq_getattr` and `mq_setattr` are implemented on the top of the `mq_getsetattr` system call. For further information about this system call the reader is encouraged to read the `mq_getsetattr(2)` manual page.

As previously noted, message priority determines the order in which messages are retrieved from the queue. Messages with higher priority values are delivered before those with lower priorities. POSIX message queues operate as priority-based queues rather than simple FIFO queues.

The `mqsndmsg.c` and `mqrvcvmsg.c` programs allows the user to send messages with different priorities. `mqsndmsg.c` takes from the command-line the name of the queue, the message and the priority.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <mqueue.h>
5 #include <fcntl.h>
6 #include <err.h>

7 #define MAX_SIZE 128

8 int main(int argc, char *argv[]) {
9     mqd_t mq;
10    struct mq_attr attr;
11    char *message;

12    if (argc != 4) {
13        fprintf(stderr, "Usage: %s <mq-name> <msg> <pri> \n",
14                argv[0]);
15        exit(EXIT_FAILURE);
16    }

17    attr.mq_flags = 0;
18    attr.mq_maxmsg = 10;
19    attr.mq_msgsize = MAX_SIZE;
20    attr.mq_curmsgs = 0;

21    message = argv[2];
22    printf("%s %d", message, strlen(message));

23    mq = mq_open(argv[1], O_CREAT | O_WRONLY, 0644, NULL);
24    if (mq == (mqd_t) -1) {
25        perror("mq_open");
26        exit(1);
27    }
28    if (mq_send(mq, message, strlen(message) + 1, atoi(argv[3])) == -1) {
29        perror("mq_send");
30    }
```

```

29     exit(1);
30 }
31 mq_close(mq);
32 return 0;
33 }

```

mqsndmsg.c Sender

After setting the attributes, the program creates/opens the queue, sends the message and closes the queue.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mqueue.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>

6  int main(int argc, char *argv[]) {
7      mqd_t mq;
8      void *buffer;
9      unsigned int priority;
10     struct mq_attr attr;

11     if (argc != 2) {
12         fprintf(stderr, "\nUsage: %s <mq-name>", argv[0]);
13         exit(EXIT_FAILURE);
14     }

15     mq = mq_open(argv[1], O_RDONLY, 0644, &attr);
16     if (mq == (mqd_t) -1) {
17         perror("mq_open");
18         exit(1);
19     }

20     if (mq_getattr(mq, &attr) == -1) {
21         perror("mq_getattr");
22         exit(EXIT_FAILURE);
23     }

24     buffer = malloc(attr.mq_msgsize);
25     if (buffer == NULL) {
26         perror("malloc");
27         exit(EXIT_FAILURE);
28     }
29     if (mq_receive(mq, buffer, attr.mq_msgsize, &priority) == -1) {
30         perror("mq_receive");
31         exit(1);
32     }

```

```
33     printf("Received message: %s ", buffer);
34     printf("Priority: %u\n", priority);
35     mq_close(mq);
36     free(buffer);
37     return 0;
38 }
```

mqrvcmsg.c Receiver

The receiver opens the queue at line 15, and retrieves the attributes at line 20 using `mq_getattr`. Next, at line 24, it allocates memory for the buffer using the `mq_msgsize` field of the `attr` structure.

At line 29, it reads the message from the queue using `mq_receive`.

In lines 35-36, the program closes the queue and frees the memory.

Let's execute.

```
[linux@ipc] ./mqsndmsg /newqu "msg-a" 5
msg-a 5

[linux@ipc] ./mqsndmsg /newqu "msg-b" 1
msg-b 5

[linux@ipc] ./mqsndmsg /newqu "msg-c" 10
msg-c 5

[linux@ipc] ./mqsndmsg /newqu "msg-d" 3
msg-d 5

[linux@ipc] ./mqrvcmsg /newqu
Received message: msg-c Priority: 10

[linux@ipc] ./mqrvcmsg /newqu
Received message: msg-a Priority: 5

[linux@ipc] ./mqrvcmsg /newqu
Received message: msg-d Priority: 3

[linux@ipc] ./mqrvcmsg /newqu
Received message: msg-b Priority: 1
```

Messages are read from the queue based on their priority.

VI.2.4 Message queue notification: `mq_notify`

A process can register to receive asynchronous notifications when a message arrives in the message queue. The `mq_notify`¹⁰ function allows the calling process to register or unregister for delivery of an asynchronous notification.

```
#include <mqueue.h>
#include <signal.h>
```

```
int mq_notify(mqd_t mqdes, const struct sigevent *sevp);
```

Returns: on success 0, on error -1 and errno

`mqdes` is the message queue descriptor.

The `sevp` argument is a pointer to a `sigevent` structure. If this parameter is a non-null pointer, `mq_notify` registers the process to receive notifications. The `sigev_notify` of `sevp` specifies how notification is performed. It can have one of the following value:

`SIGEV_NONE`

A "null" notification: the calling process is registered as the target for notification, but when a message arrives, no notification is sent

`SIGEV_SIGNAL`

Notify the process by sending the signal specified in `sigev_signo`. The `si_code` field of the `siginfo_t` structure will be set to `SI_MESGQ`. In addition, `si_pid` will be set to the PID the process that sent the message, and `si_uid` will be set to the real user ID of the sending process

`SIGEV_THREAD`

Upon message delivery, invoke `sigev_notify_function` as if it were the start function of a new thread

Only one process can be registered to receive notification from a message queue.

If `sevp` is `NULL` and the calling process is currently registered to receive notifications for the message queue, the registration is removed. This allows another process to register for message notifications on the same queue.

A notification is generated only when a new message arrives in an empty queue. If the queue is not empty at the time `mq_notify` is called, a notification will be triggered only after the queue has been emptied and a new message arrives.

If another process or thread is already blocked in `mq_receive` waiting for a message from an empty queue, the notification mechanism is bypassed. In this case, the message is delivered directly to the waiting process or thread, and the notification registration remains active.

Notifications are delivered only once. After a notification is triggered, the registration is automatically removed, allowing another process to register. If the same process wishes to receive further notifications, it must call `mq_notify` again. This re-registration should typically occur before all pending messages are consumed. Using nonblocking mode can help empty the queue without blocking when no messages remain.

¹⁰ See the `mq_notify(3)` manual page

VI.2.5 Notification via thread

The `mqnotify.c` program is taken from the `mq_notify(3)` manual page, it registers a notification request for the message queue specified as a command-line argument. Notification is implemented by creating a thread, which executes a handler function that reads one message from the queue and then terminates the process.

```
1 #include <err.h>
2 #include <mqueue.h>
3 #include <pthread.h>
4 #include <signal.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <unistd.h>

8 static void tfunc(union sigval sv)
9 {
10     struct mq_attr attr;
11     ssize_t nr;
12     void *buf;
13     mqd_t mqdes = *((mqd_t *) sv.sival_ptr);

14     if (mq_getattr(mqdes, &attr) == -1)
15         err(EXIT_FAILURE, "mq_getattr");
16     buf = malloc(attr.mq_msgsize);
17     if (buf == NULL)
18         err(EXIT_FAILURE, "malloc");

19     nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
20     if (nr == -1)
21         err(EXIT_FAILURE, "mq_receive");

22     printf("Read %zd bytes from MQ\n", nr);
23     free(buf);
24     exit(EXIT_SUCCESS);
25 }

26 int main(int argc, char *argv[])
27 {
28     mqd_t mqdes;
29     struct sigevent sev;

30     if (argc != 2) {
31         fprintf(stderr, "Usage: %s <mq-name>\n", argv[0]);
32         exit(EXIT_FAILURE);
33     }
```

```

34  mqdes = mq_open(argv[1], O_RDONLY);
35  if (mqdes == (mqd_t) -1)
36      err(EXIT_FAILURE, "mq_open");
37  sev.sigev_notify = SIGEV_THREAD;
38  sev.sigev_notify_function = tfunc;
39  sev.sigev_notify_attributes = NULL;
40  sev.sigev_value.sival_ptr = &mqdes;
41  if (mq_notify(mqdes, &sev) == -1)
42      err(EXIT_FAILURE, "mq_notify");

43  pause();
44  }

```

mqnotify.c Notification

At line 29, a `struct sigevent` variable is declared. This structure is used to specify how the process will be notified when a message arrives in the queue.

Next, the program opens the message queue using:

```

34  mqdes = mq_open(argv[1], O_RDONLY);

```

This call opens an existing message queue in read-only mode. It assumes that the queue has already been created; otherwise, the call fails.

In lines 37–40, the `sigevent` structure is initialized:

- `sigev_notify` is set to `SIGEV_THREAD`, indicating that notification will be delivered by creating a new thread
- `sigev_notify_function` is set to `tfunc`, which will be executed when a message arrives
- `sigev_notify_attributes` is set to `NULL`, meaning default thread attributes are used
- `sigev_value.sival_ptr` is set to the address of `mqdes`, allowing the thread function to access the queue descriptor

The program then registers for notification using:

```

41  if (mq_notify(mqdes, &sev) == -1)

```

This call associates the notification request with the queue. When a message arrives on an empty queue, the kernel will create a new thread and invoke `tfunc`.

The call to `pause` at line 43 suspends the main thread indefinitely, waiting for a signal. In practice, the process will terminate when the notification thread calls `exit`.

The function `tfunc` is executed in a new thread when a notification is triggered.

At line 13, the queue descriptor is retrieved from the `sigevent` structure:

```

13  mqd_t mqdes = *((mqd_t *) sv.sival_ptr);

```

The `sival_ptr` field is cast to a pointer to `mqd_t`, and its value is dereferenced to obtain the queue descriptor.

Next, the function retrieves the queue attributes:

```
14    if (mq_getattr(mqdes, &attr) == -1)
```

This is necessary to determine the maximum message size (`mq_msgsize`), which is used to allocate a suitable buffer.

At line 16, memory is allocated dynamically with `malloc`

```
16    buf = malloc(attr.mq_msgsize);
```

This ensures the buffer is large enough to hold any message from the queue.

The function then calls:

```
19    nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
```

This retrieves a message from the queue. In this context, the call should not block because the notification is only triggered when a message is available.

At line 22, the number of bytes read is printed.

At line 23, the allocated buffer is freed.

At line 24, `exit(EXIT_SUCCESS)` is called to terminate the entire process including the main thread.

To send a message to this program, we can use the `mq_send.c` program. We add the `O_CREAT` flag to the `mq_open` call of `mq_send.c` as follows

```
11    mq = mq_open(QUEUE_NAME, O_CREAT | O_WRONLY, 0644, &attr);
```

Let's execute

Shell 1

```
[linux@ipc] ./mq_send /myqueue  
Enter message:
```

Shell 2

```
[linux@ipc] ./mqnotify /myqueue
```

Shell 1

```
Enter message: hello notification program!!
```

Shell 2

```
Read 30 bytes from MQ
```

The execution shows that when a message is sent to an empty queue, the registered notification is triggered, causing the thread function to execute and read the message.

The program demonstrates *asynchronous notification* using `mq_notify` with `SIGEV_THREAD`, where message arrival triggers the execution of a handler function in a new thread. Notification occurs only when a message arrives in an empty queue, ensuring that the handler is invoked exactly once per registration.

VI.2.6 Notification via signal

The `mqsig.c` program demonstrates *asynchronous notification* using `mq_notify` with `SIGEV_SIGNAL`, where message arrival triggers the delivery of a signal. The *signal handler* is responsible for reading the message from the queue and terminating the process. As with all `mq_notify` usage, the notification is delivered only once and only when a message arrives in an empty queue.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mqueue.h>
4 #include <signal.h>
5 #include <fcntl.h>
6 #include <unistd.h>

7 #define QUEUE_NAME "/myqueue"
8 mqd_t mqdes;

9 void handler(int sig) {
10     struct mq_attr attr;
11     char *buf;
12     ssize_t nr;

13     printf("Signal received: message available\n");
14     if (mq_getattr(mqdes, &attr) == -1) {
15         perror("mq_getattr");
16         exit(1);
17     }

18     buf = malloc(attr.mq_msgsize);
19     if (buf == NULL) {
20         perror("malloc");
21         exit(1);
22     }

23     nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
24     if (nr == -1) {
25         perror("mq_receive");
26         exit(1);
27     }
```

```

28     printf("Received message (%zd bytes): %s\n", nr, buf);
29     free(buf);
30     exit(0);
31 }

32 int main() {
33     struct sigevent sev;
34     struct sigaction sa;
35     mqdes = mq_open(QUEUE_NAME, O_RDONLY);
36     if (mqdes == (mqd_t)-1) {
37         perror("mq_open");
38         exit(1);
39     }

40     sa.sa_handler = handler;
41     sigemptyset(&sa.sa_mask);
42     sa.sa_flags = 0;
43     if (sigaction(SIGUSR1, &sa, NULL) == -1) {
44         perror("sigaction");
45         exit(1);
46     }

47     sev.sigev_notify = SIGEV_SIGNAL;
48     sev.sigev_signo = SIGUSR1;
49     if (mq_notify(mqdes, &sev) == -1) {
50         perror("mq_notify");
51         exit(1);
52     }
53     printf("Waiting for message...\n");
54     pause();
55     return 0;
56 }

```

mqsig.c Notification via signal

The function handler is a signal handler that is invoked when a notification signal is delivered.

At line 14, the handler retrieves the attributes of the message queue using `mq_getattr`. This is necessary to determine the maximum message size.

At line 18, memory is dynamically allocated for the message buffer using `malloc`, with a size equal to `mq_msgsize`.

The message is then read from the queue using

```

23     nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);

```

This call retrieves one message from the queue. In this context, it should not block, since the signal is delivered only when a message is available.

At lines 29–30, the allocated memory is freed, and the process is terminated using `exit`. As a result, the program handles a single message and then exits.

In `main`, at lines 33–34, two structures are declared:

- `struct sigevent sev`, used to specify the notification mechanism
- `struct sigaction sa`, used to define the signal handler

At line 35, the program opens an existing message queue using

```
35    mqdes = mq_open(QUEUE_NAME, O_RDONLY);
```

This assumes that the queue has already been created by another process.

At lines 40–43, the signal handler is installed using `sigaction`. The handler function `handler` is associated with the signal `SIGUSR1`.

At lines 47–48, the notification mechanism is configured:

- `sigev_notify` is set to `SIGEV_SIGNAL`, meaning that a signal will be sent when a message arrives
- `sigev_signo` is set to `SIGUSR1`, the signal that will trigger the handler

The program then registers for notification using

```
49    if (mq_notify(mqdes, &sev) == -1) {
```

This call requests that the process be notified when a message arrives on an empty queue.

Finally, at line 54, the call to `pause` suspends the process until a signal is received. When a message arrives, the signal is delivered, the handler is executed, and the process terminates.

This program is intended for demonstration purposes. It is important to note that functions such as `printf`, `malloc`, and `mq_receive` are not guaranteed to be async-signal-safe, and therefore should not be used inside a signal handler in production code.

To send a message we can use the `mq_send.c` program.

Let's execute

Shell 1

```
[linux@ipc] ./mq_send
Enter message:
```

Shell 2

```
[linux@ipc] ./mqsig  
Waiting for message...
```

Shell 1

```
Enter message: Hello notification by signal  
Message sent: Hello notification via signal
```

Shell 2

```
Waiting for message...  
Signal received: message available  
Received message (31 bytes): Hello notification via signal
```

The execution demonstrates that, since `sev.sigev_notify` is set to `SIGEV_SIGNAL`, when the message is received, `SIGUSR1` is delivered and the handler invokes `mq_receive`.

A safer alternative is to use `sigwait` which allows signals to be handled synchronously and avoids the limitations of signal handlers. The `mqsigwait.c` program demonstrates a safe pattern for *asynchronous notification* using `mq_notify` with `SIGEV_SIGNAL` combined with `sigwait`. By blocking the signal and handling it synchronously, the program avoids the limitations of traditional signal handlers and can safely use *standard library functions* such as `malloc` and `printf`.

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3 #include <mqueue.h>  
4 #include <signal.h>  
5 #include <fcntl.h>  
6 #include <unistd.h>  
  
7 #define QUEUE_NAME "/myqueue"  
  
8 int main() {  
9     mqd_t mqdes;  
10    struct sigevent sev;  
11    struct mq_attr attr;  
12    sigset_t mask;  
13    int sig;  
  
14    sigemptyset(&mask);  
15    sigaddset(&mask, SIGUSR1);  
16    if (sigprocmask(SIG_BLOCK, &mask, NULL) == -1) {  
17        perror("sigprocmask");  
18        exit(1);  
19    }
```

```

20  mqdes = mq_open(Queue_NAME, O_RDONLY);
21  if (mqdes == (mqd_t)-1) {
22      perror("mq_open");
23      exit(1);
24  }

25  sev.sigev_notify = SIGEV_SIGNAL;
26  sev.sigev_signo = SIGUSR1;
27  if (mq_notify(mqdes, &sev) == -1) {
28      perror("mq_notify");
29      exit(1);
30  }

31  printf("Waiting for message...\n");
32  if (sigwait(&mask, &sig) != 0) {
33      perror("sigwait");
34      exit(1);
35  }

36  printf("Signal received: message available\n");
37  if (mq_getattr(mqdes, &attr) == -1) {
38      perror("mq_getattr");
39      exit(1);
40  }

41  char *buf = malloc(attr.mq_msgsize);
42  if (buf == NULL) {
43      perror("malloc");
44      exit(1);
45  }

46  ssize_t nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
47  if (nr == -1) {
48      perror("mq_receive");
49      exit(1);
50  }

51  printf("Received message (%zd bytes): %s\n", nr, buf);
52  free(buf);
53  mq_close(mqdes);
54  return 0;
55 }

```

mqsigwait.c Safer notification via signal

At line 14, the program initializes an empty signal set using `sigemptyset`.
At line 15, the signal `SIGUSR1` is added to this set using `sigaddset`.

In lines 16–19, the program blocks `SIGUSR1` using `sigprocmask`. This is essential because it ensures that the signal will not be delivered asynchronously, but instead can be handled synchronously via `sigwait`.

At line 20, the program opens an existing message queue

```
20    mqdes = mq_open(QUEUE_NAME, O_RDONLY);
```

This assumes that the queue has already been created by another process.

In lines 25–26, the `sigevent` structure is configured by setting `sigev_notify` to `SIGEV_SIGNAL`, and `sigev_signo` to `SIGUSR1`.

The program then registers for notification using

```
27    if (mq_notify(mqdes, &sev) == -1) {
```

At line 31, the program prints a message indicating that it is waiting for input.

At line 32, the program calls

```
32    if (sigwait(&mask, &sig) != 0) {
```

This call suspends execution until one of the signals in `mask`, in this case `SIGUSR1`, is delivered. Since the signal is blocked, it is not handled asynchronously but instead causes `sigwait` to return. Once `sigwait` returns, the program continues execution in a safe context.

At line 37, the program retrieves the queue attributes using `mq_getattr`. This is necessary to determine the maximum message size.

At line 41, memory is dynamically allocated for the message buffer using `malloc`, with a size equal to `mq_msgsize`.

At line 46, the program reads a message from the queue using

```
46    ssize_t nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
```

This call retrieves one message from the queue. Since the signal was triggered by message arrival, the call is expected to succeed without blocking.

Finally at line 52, the allocated memory is freed and at line 53, the queue descriptor is closed using `mq_close`.

The combination of `SIGEV_SIGNAL` and `sigwait` is the recommended safe pattern for signal-based notification. The signal must be blocked before calling `mq_notify`, otherwise it may be delivered before `sigwait` is ready to receive it.

Let's execute

Shell 1

```
[linux@ipc] ./mq_send
```

Enter message:

Shell 2

[\[linux@ipc\]](#) ./mqsigwait

Waiting for message...

Shell 1

Enter message: hello from sender

Message sent: hello from sender

Shell 2

Waiting for message...

Signal received: message available

Received message (19 bytes): hello from sender

When the message is received, the signal is generated, and this causes `sigwait` to return. The program then retrieves the message from the queue and prints its contents.

The competition

Other books that cover this subject are listed with a brief description.

Advanced Programming in the UNIX Environment, W. by Richard Stevens & Stephen A. Rago 3ed, Addison-Wesely, 2013

Advanced Programming in the UNIX Environment by Richard W. Stevens is a classic reference on UNIX system programming in C. It provides a detailed and practical exploration of POSIX and UNIX system interfaces, focusing on how operating systems expose low-level functionality to applications.

The book covers core topics such as file I/O, processes, signals, interprocess communication, and terminal control, supported by clear explanations and extensive code examples. It is widely regarded as a foundational text for understanding and working with UNIX and UNIX-like systems at the system level.

The Linux Programming Interface, by Michael Kerrisk, No Starch Press, 2010

The Linux Programming Interface by Michael Kerrisk is a comprehensive and authoritative guide to system programming on Linux. It provides an extensive and detailed explanation of Linux system calls and library interfaces, closely aligned with POSIX standards and real kernel behavior.

The book covers a wide range of topics, including file I/O, process and thread management, signals, interprocess communication, memory allocation, and networking interfaces. Each concept is supported by clear explanations, practical examples, and well-structured sample programs.

Widely regarded as one of the most complete references for Linux system programming, it is both a learning resource and a definitive guide for developers working close to the operating system.

Linux System Programming, by Robert Love, O'Reilly, 2013

Linux System Programming by Robert Love is a practical introduction to system-level programming on Linux. It focuses on how applications interact with the Linux kernel through system calls and core APIs, emphasizing real-world usage rather than theoretical detail.

The book covers essential topics such as file I/O, process management, signals, timers, and basic interprocess communication. It also provides insight into how Linux handles processes, memory, and system resources.

Designed as an accessible guide for developers, it is a solid starting point for understanding Linux system programming.

Linux System Programming Techniques, by Jack-Benny Persson, Packt Publishing, 2021

Linux System Programming Techniques by Jack-Benny Persson is a practical guide focused on advanced techniques for developing system-level software on Linux. It emphasizes a hands-on approach to understanding how applications interact with the Linux kernel through system calls and core interfaces.

The book covers key areas such as process management, file I/O, interprocess communication, signals, and threading, with an emphasis on practical implementation and real-world usage patterns.

Contact Information

Full name: Marco Bruno

Address: Viale Giustiniano Imperatore 274, 00145, Rome, Italy

Email: mr.marco.bruno.2022@gmail.com

Tel: +855 963741727